

Python을 이용한 파일 처리

05 사무용 문서 다루기

목차 05 사무용 문서 다루기

1. pypdf
2. python-docx
3. openpyxl
4. python-pptx

1. pypdf

- Python을 이용한 PDF 문서 조작 라이브러리
- 다음과 같은 작업을 할 수 있음
 - PDF 문서 작성
 - PDF 문서에서 텍스트 및 이미지 추출하기
 - PDF 파일 합치기/나누기
 - 암호 설정



간단한 예제(PDF 문서에서 텍스트 추출)

Python을 이용한 파일
처리

05 사무용 문서 다루기

1. pypdf

English:
Hello World

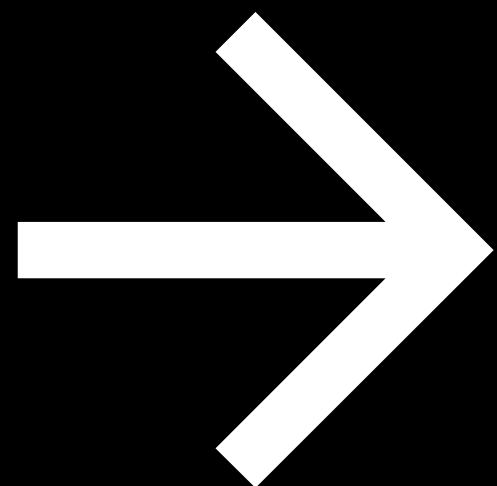
Arabic:
مرحبا بالعالم

Russian:
Привет, мир

Chinese (traditional):
你好世界

Thai:
สวัสดีชาวโลก

Japanese:
こんにちは世界



English:
Hello World

Arabic:
بالعالم مرحبا

Russian:
Привет, мир

Chinese (traditional):
你好世界

Thai:
สวัสดีชาวโลก

Japanese:
こんにちは世界

다음과 같은 코드를 이용하여 PDF 문
서에서 텍스트만 추출할 수 있음

```
from pypdf import PdfReader

text = ""
reader = PdfReader("hello-world.pdf")
for page in reader.pages:
    text += page.extract_text()
print(text)
```

- 기본적인 기능만 설치하는 경우
 - Windows: `py -m pip install pypdf`
 - macOS: `python3 -m pip install pypdf`
- 이미지 다루기 및 암호화 등을 하는 경우에는 다음과 같이 설치
 - Windows: `py -m pip install pypdf[full]`
 - macOS: `python3 -m pip install pypdf[full]`

```
from pypdf import PdfReader, PdfWriter

FILENAME_INPUT = "example.pdf"
FILENAME_OUTPUT = "example-write.pdf"

# PdfReader로 PDF 파일 열기
reader = PdfReader(FILENAME_INPUT)
pages = reader.pages

# PdfWriter로 빈 파일 쓰기
with PdfWriter() as writer:
    writer.write(FILENAME_OUTPUT)
```

- pypdf의 대표적인 두 class
 - PdfReader
 - PdfWriter
- PdfReader
 - PDF 파일에서 진행하는 모든 종류의 읽기 작업에 사용
 - 문자, 텍스트 추출, 주석 불러오기 등
 - 파일명을 인자로 넘겨 객체 초기화
 - `pages` 속성으로 각 페이지를 선택할 수 있음(리스트)
- PdfWriter
 - PDF 파일에서 진행하는 모든 종류의 쓰기 작업에 사용
 - 합치기, 자르기, 변형, 암호화, 복호화 등
 - 추가 전달 인자 없이 객체 초기화



가장 간단한 예제

```
from pypdf import PdfReader
```

```
reader = PdfReader("hello-world.pdf")
```

```
for page in reader.pages:
```

```
    print(page.extract_text())
```

- PdfReader의 각 page의 메서드 **extract_text** 사용



```
from pypdf import PdfReader

reader = PdfReader("document.pdf")

page = reader.pages[0]

# 이미지 파일 선택
image_file_object = page.images[0]

# 이미지 파일 이름은 "name" 속성으로 고정
print(image_file_object.name)
# Image81.png

# 추출을 위해 "data" 속성을 이용
with open(image_file_object.name, "wb") as fp:
    fp.write(image_file_object.data)
```

- PDF 이미지는 PdfReader의 각 page의 속성 **images**가 가리키고 있음
- 해당 속성을 반복하여 파일에 쓰는 형태로 추출 가능
- **name** 속성이 파일의 원래 이름 및 확장자를 가지고 있음
 - 하나의 PDF 안에 있는 이미지 파일의 이름은 고유하지 않
으므로 주의
- **data** 속성이 이미지 데이터를 가지고 있음
 - 바이너리 모드로 파일에 쓰면 이미지 파일로 저장됨

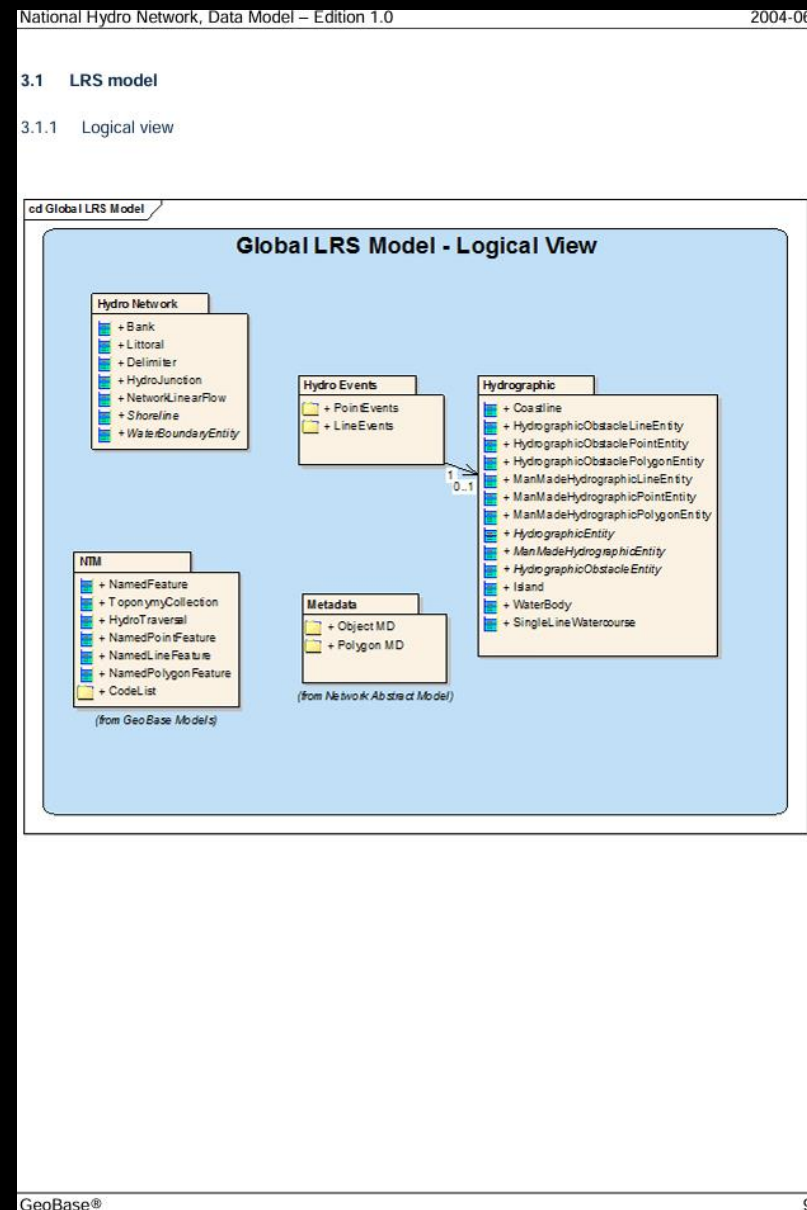


이미지 추출 예제

Python을 이용한 파일
처리

05 사무용 문서 다루기

1. pypdf



원본 페이지

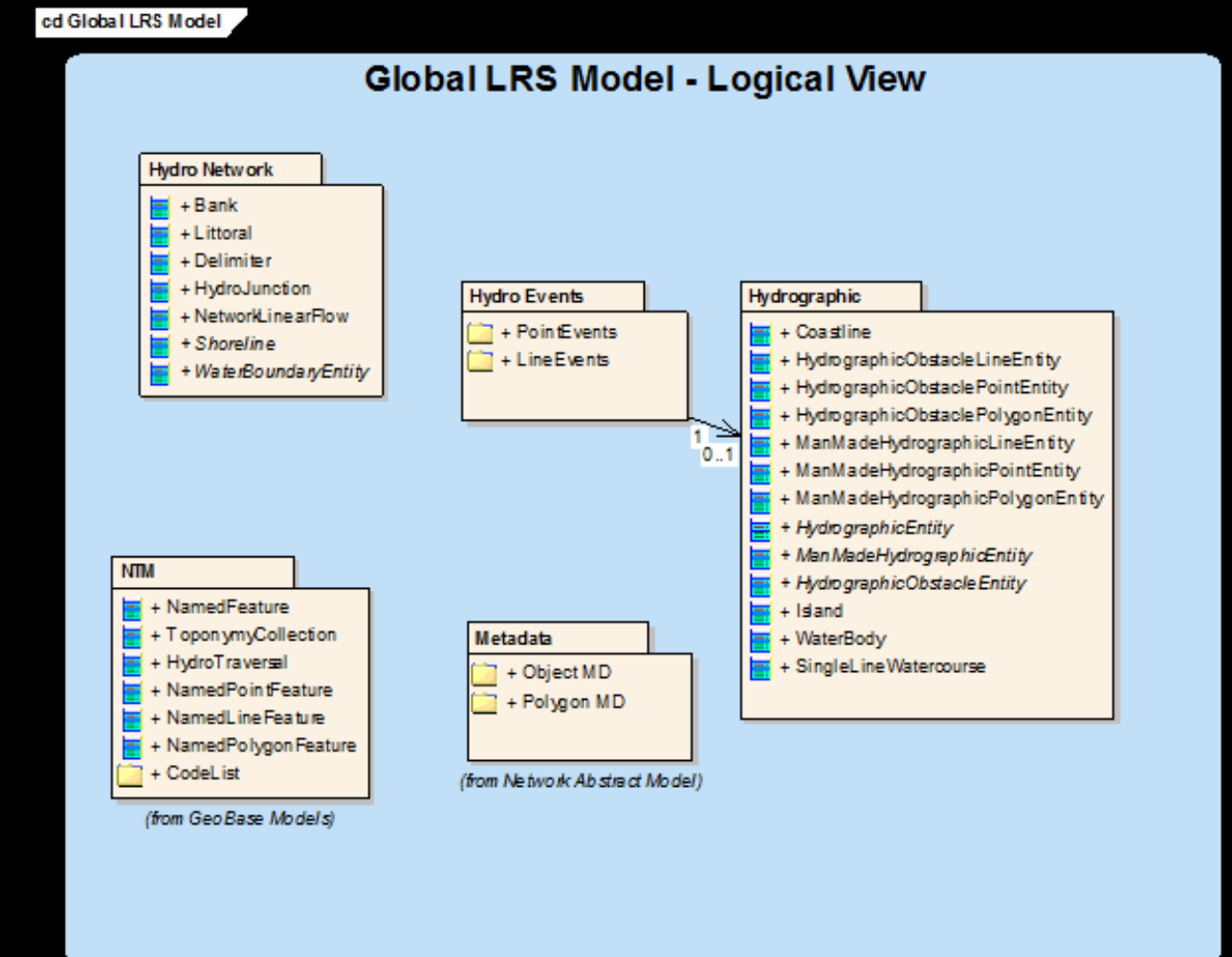
```
from pypdf import PdfReader

reader = PdfReader("GeoBase_NHNC1_Data_Model_UML_EN.pdf")

page = reader.pages[8]

# enumerate: 반복 가능한 요소에 번호를 매기는 함수
for i, image_file_object in enumerate(page.images):
    # 각 파일의 이름이 고유하지 않을 수 있으므로 번호를 매김
    with open(f"{i}_{image_file_object.name}", "wb") as fp:
        fp.write(image_file_object.data)
```

이미지 추출 스크립트



추출된 이미지



```
"""document1.pdf와 document2.pdf를 순서대로 합쳐 내보내기"""  
  
from pypdf import PdfWriter  
  
with PdfWriter() as writer:  
    for pdf in ["document1.pdf", "document2.pdf"]:  
        writer.append(pdf)  
    writer.write("merged.pdf")
```

- **PdfWriter**를 사용하여 여러 PDF 파일을 하나의 PDF 파일로 합칠 수 있음
 - 이전 PDF 파일의 마지막 페이지의 끝에 새로운 PDF 파일이 추가되는 형태
- 리스트처럼 **append**를 이용하여 합칠 파일을 추가할 수 있음
- 다음 형태의 인자 지원
 - 파일 경로
 - 파일 객체
 - PdfReader 객체



```
from pypdf import PdfWriter

with PdfWriter() as writer:
    writer.append("document1.pdf")
    writer.append("document2.pdf")
    # 지금까지 writer에 쌓인 PDF 문서의 두 번째 페이지 뒤에 document3.pdf 삽입
    writer.merge(2, "document3.pdf")

writer.write("merged.pdf")
```

- **merge** 메서드를 이용하여 문서의 삽입 위치 설정 가능
- 리스트의 **insert** 메서드와 비슷한 동작을 수행
- 유의: 위치는 파일 단위가 아닌 페이지 단위



```
from pypdf import PdfWriter

with PdfWriter() as writer:
    # invoice.pdf의 첫 번째 페이지만 추가
    writer.append("invoice.pdf", pages=(0, 1))
    # terms.pdf의 3-4 페이지 추가
    writer.append("terms.pdf", pages=(2, 4))
    # 첫 페이지 위치에 cover-art.pdf의 세 번째 페이지 추가
    writer.merge(0, "cover-art.pdf", pages=(2, 3))

writer.write("contract.pdf")
```

- **pages** 매개 변수를 사용하여 파일에서 원하는 페이지만 추가할 수 있음
- 전달 인자는 range 함수와 같음
 - 첫 페이지 번호를 0으로 간주
 - 원하는 시작 페이지부터 끝 페이지까지 지정할 수 있음
 - 끝 페이지 번호는 추가되지 않음
 - 필요한 경우 step(세 번째 값) 지정 가능



```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("document.pdf")

with PdfWriter() as writer:
    # 4 페이지만 추가
    writer.add_page(reader.pages[3])
    # 첫 번째 순서에 1 페이지만 추가
    writer.insert_page(reader.pages[0], 0)

    writer.write("output.pdf")
```

- 단일 페이지만 추가하거나 PdfReader 객체 대신 PageObject만 추가하려는 경우 다음 메서드를 대신 사용 가능
 - **add_page**: append와 동일한 동작
 - **insert_page**: merge와 동일한 동작, 인자의 순서가 다름



```
from pathlib import Path

from pypdf import PdfWriter

with PdfWriter() as writer:
    for pdf in sorted(Path("target").glob("*.pdf")):
        writer.append(pdf)
    writer.write("merged.pdf")
```

- target 디렉터리의 모든 PDF 파일을 사전순으로 모두 합쳐 보내내기



```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("document.pdf")

with PdfWriter() as writer:
    writer.append(reader)
    # 암호 추가
    writer.encrypt("password", algorithm="AES-256-R5")
    writer.write("output.pdf")
```

- 기존 문서 내용을 PdfWriter 객체로 복사 후 PdfWriter의 **encrypt** 메서드를 사용하여 암호화 가능
- 암호는 항상 영문자 및 특수문자만 사용해야 함
- **algorithm** 매개 변수를 통해 암호화 알고리즘 설정
 - 보안을 위해 "AES-256-R5" 사용



```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("document.pdf")

# 암호 해제
reader.decrypt("password")

with PdfWriter() as writer:
    writer.append(reader)
    writer.write("output.pdf")
```

- 기존 문서 내용을 PdfWriter 객체로 복사하기 전 PdfReader의 **decrypt** 메서드를 사용하여 암호 해제 가능

2. python-docx



- Python을 이용한 Microsoft의 Word 파일 조작 라이브러리
- Open XML 문서(.docx) 형식의 파일 읽기 및 쓰기 가능
- 기존 파일을 바탕으로 간단한 보고서 등을 작성하는 데 유리



- 설치

- Windows: `py -m pip install python-docx`
- macOS: `python3 -m pip install python-docx`

- 임포트

- `import docx`
- python-docx가 아님에 유의



간단한 문서 작성 예제

Python을 이용한 파일
처리

05 사무용 문서 다루기

2. python-docx

```
from docx import Document

document = Document()

document.add_heading("제목", level=0)

p = document.add_paragraph("본문")
p.add_run(" 해당 단락에 텍스트 추가")

p2 = document.add_paragraph()
p2.add_run("굵게").bold = True

document.save("demo.docx")
```

Python 프로그램

제목↵

본문 해당 단락에 텍스트 추가↵

굵게↵

생성된 demo.docx 파일

- Document

- 문서, 하나의 Python 파일에서 여러 개의 문서를 다룰 수 있음

- Paragraph

- 문단, 스타일을 가질 수 있으며 여러 종류의 Document 메서드로 생성 가능

- `add_heading`: 제목, `level` 매개 변수에 0-9의 값을 매겨 제목의 크기 조절 가능
- `add_page_break`: 빈 문단(줄 바꾸기에 사용)
- `add_paragraph`: 기본적인 문단, 스타일을 적용할 수 있음

- Document의 `paragraphs` 속성으로 전체 Paragraph에 접근 가능

- Table

- 표, Document의 `add_table` 메서드로 생성 가능

- Document의 `tables` 속성으로 전체 Table에 접근 가능

```
from docx import Document

for style in Document().styles:
    print(style.name)
```

- 각 문단에 적용 가능한 서식
- 글꼴을 바꾸는 것 이외에도 다음 서식이 존재
 - 제목
 - 강조
 - 목록
- 기존 문서를 불러오는 경우 해당 문서에서 작성한 스타일을 사용할 수 있음
- 기본 스타일 목록은 왼쪽 코드로 확인 가능

```
from docx import Document

document = Document()

p = document.add_paragraph("기본 문단 ")
p.add_run("굵게 ").bold = True
p.add_run("기울임꼴 ").italic = True
p.add_run("밑줄").underline = True

document.save("demo.docx")
```

기본 문단 **굵게** *기울임꼴* 밑줄

- 하나의 Paragraph에서 구분 가능한 단위
- 일반적으로 서식을 나누기 위해 사용
- Paragraph의 **runs** 속성을 통해 각 Run을 선택할 수 있음



```
from docx import Document

document = Document()

data_list = [
    ["column1", "column2", "column3"],
    [1, 2, 3],
    [4, 5, 6],
]

table = document.add_table(
    rows=0,
    cols=len(data_list[0]),
    style="Table Grid",
)

for data in data_list:
    row_cells = table.add_row().cells
    for i, item in enumerate(data):
        row_cells[i].text = str(item)

document.save("demo.docx")
```

column1↵	column2↵	column3↵
1↵	2↵	3↵
4↵	5↵	6↵

- Document의 **add_table** 메서드를 사용하여 표 추가 가능
- **rows, cols** 매개 변수를 통해 열, 행 개수 정의
- **style** 매개 변수로 표의 스타일 지정
 - 지정하지 않으면 선이 그려지지 않음
 - 기본 표 스타일: "Table Grid"

3. openpyxl

- Python을 이용한 Microsoft의 Excel 파일 조작 라이브러리
- Excel만의 기능 (스타일, 차트 등)을 활용하는 데 특화
- 데이터 처리 및 분석을 위해 pandas와 함께 쓰이는 경우가 많음

• 설치

- Windows: `py -m pip install openpyxl`
- macOS: `python3 -m pip install openpyxl`

• Excel 파일에서 이미지 파일을 다루는 경우 Pillow를 추가 설치

- Windows: `py -m pip install pillow`
- macOS: `python3 -m pip install pillow`



간단한 openpyxl 예제

Python을 이용한 파일
처리

05 사무용 문서 다루기

3. openpyxl

```
from openpyxl import Workbook
from openpyxl.chart import BarChart, Reference

wb = Workbook()
ws = wb.active

data = [
    ["rowcol", "column 1", "column2"],
    ["row 1", 2, 3],
    ["row 2", 5, 6],
]

for row in data:
    ws.append(row)

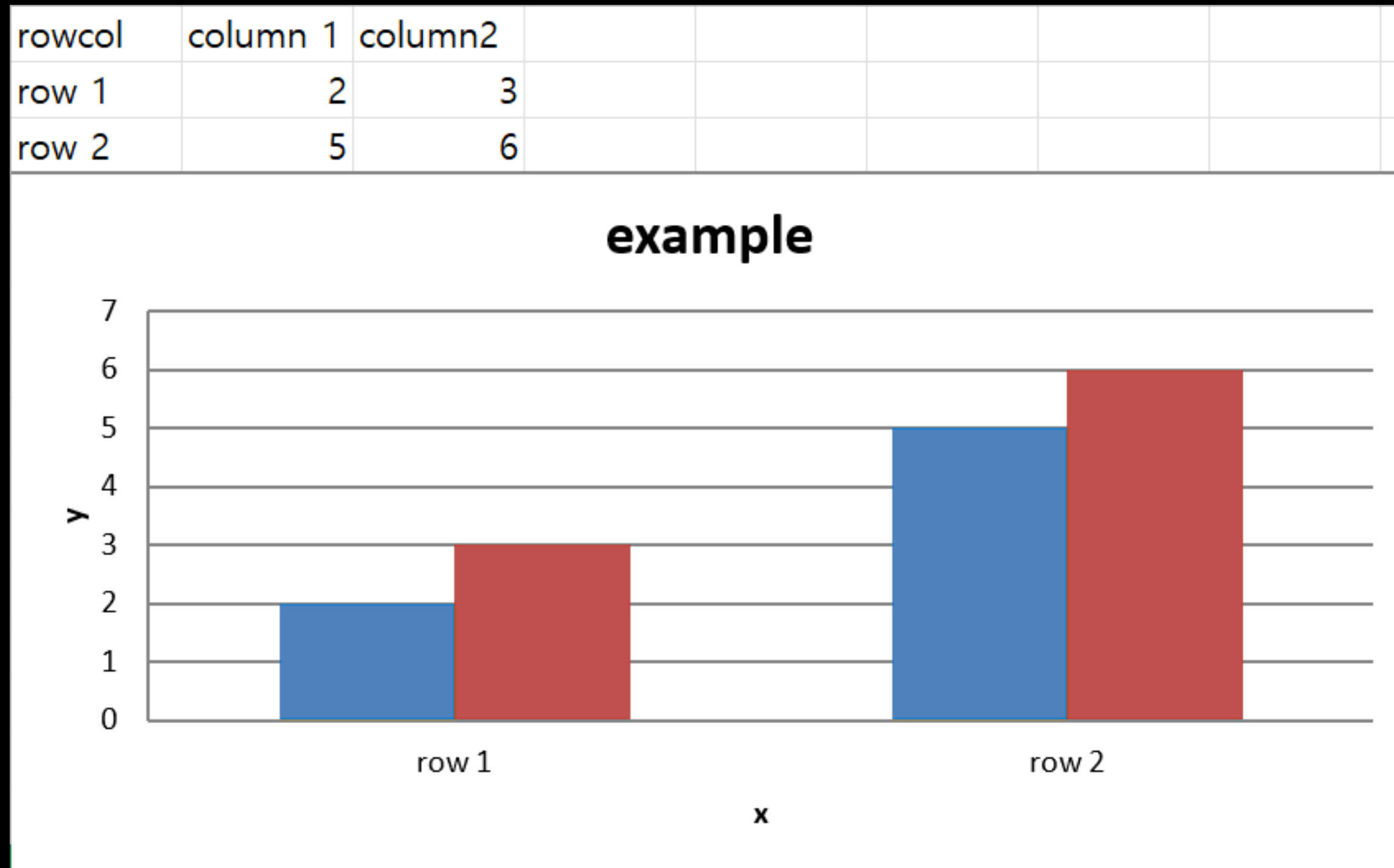
chart = BarChart()
chart.type = "col"
chart.title = "example"
chart.y_axis.title = "y"
chart.x_axis.title = "x"
chart.legend = None

data = Reference(ws, min_col=2, min_row=2, max_col=3, max_row=3)
categories = Reference(ws, min_col=1, min_row=2, max_col=1, max_row=3)

chart.add_data(data)
chart.set_categories(categories)

ws.add_chart(chart, "A4")

wb.save("document.xlsx")
```





```
from openpyxl import Workbook

# 새 문서 생성
wb = Workbook()

# 첫 번째 워크시트 불러오기
ws = wb.active

# 워크시트 이름 확인 및 변경
print(ws.title) # Sheet
ws.title = "Example"

# 두 번째 위치에 새 워크시트 만들기
# 숫자는 생략 가능하며 생략하면 기본적으로 끝에 생성
ws_new = wb.create_sheet("New Sheet", 1)

# 시트 복사하기
# 워크시트의 이름으로 선택 가능
ws_copy = wb.copy_worksheet(wb["New Sheet"])

# 존재하는 모든 시트 확인
print(wb.sheetnames) # ['Example', 'New Sheet', 'Example Copy']

# 문서 저장하기
wb.save("document.xlsx")
```

- **Workbook**: 하나의 문서를 가리키는 클래스
 - **Workbook.active**: 현재 문서의 첫 번째 워크시트
 - **Workbook.sheetnames**: 모든 워크시트의 이름을 담은 리스트
- **Workbook.create_sheet()**: 새 시트를 만드는 메서드
- **Workbook.copy_worksheet()**: 기존 워크시트를 복제하는 메서드
- **Workbook.save()**: 문서를 저장하는 메서드



Workbook 생성 방법

Python을 이용한 파일
처리

05 사무용 문서 다루기

3. openpyxl

```
from openpyxl import Workbook  
  
wb = Workbook()
```

새로운 Workbook 생성

```
from openpyxl import load_workbook  
  
wb = load_workbook("document.xlsx")
```

이미 존재하는 Excel 파일에서 Workbook 생성



빈 문서에 자료 추가하기

```
from openpyxl import Workbook

wb = Workbook()
ws = wb.active

data = [
    [1, 2],
    [3, 4],
]

for row in data:
    ws.append(row)

wb.save("document.xlsx")
```

1	2
3	4

- 빈 문서에 자료를 추가하는 경우 **append** 메서드를 이용하여 추가 가능



```
from openpyxl import Workbook

wb = Workbook()
ws = wb.active

# 셀에 접근하여 값 변경
ws["A1"] = 1
ws["B1"] = 2
ws["A2"] = 3
ws["B2"] = 4

# 슬라이싱을 이용하여 셀 순회하기
for row in ws["A1":"B2"]:
    for cell in row:
        print(cell.value)
        # 1
        # 2
        # 3
        # 4

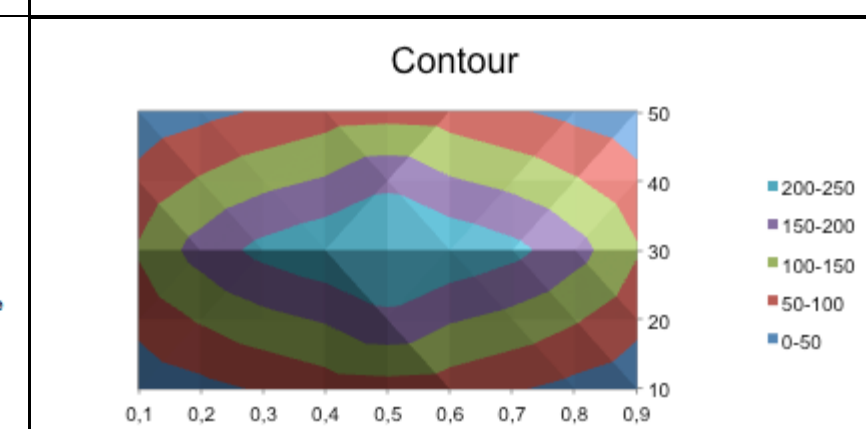
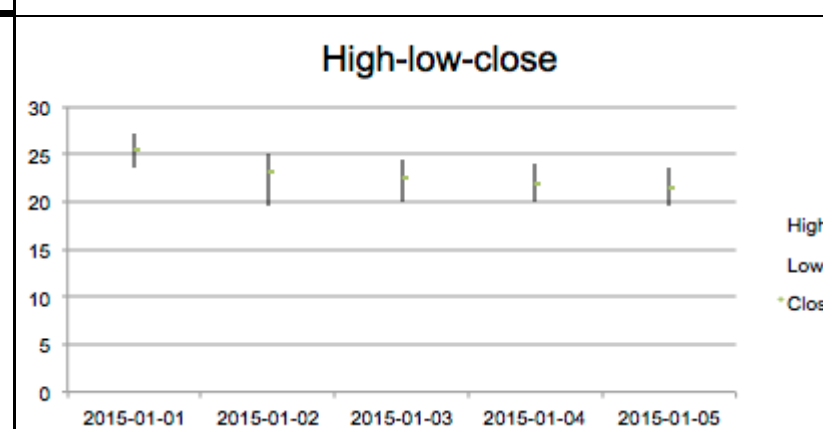
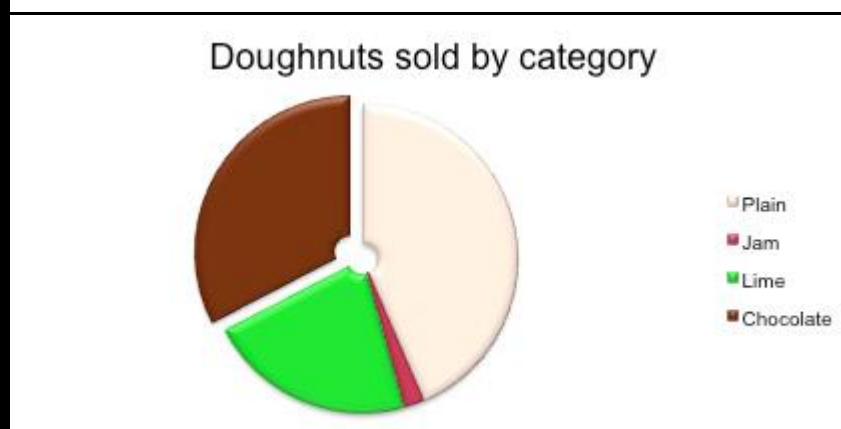
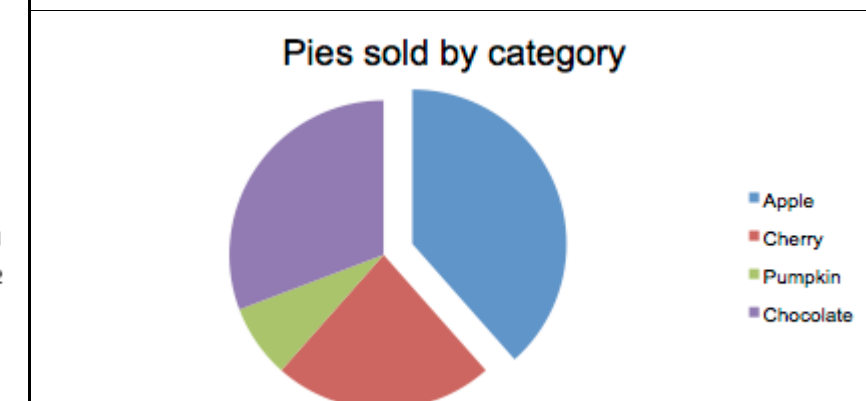
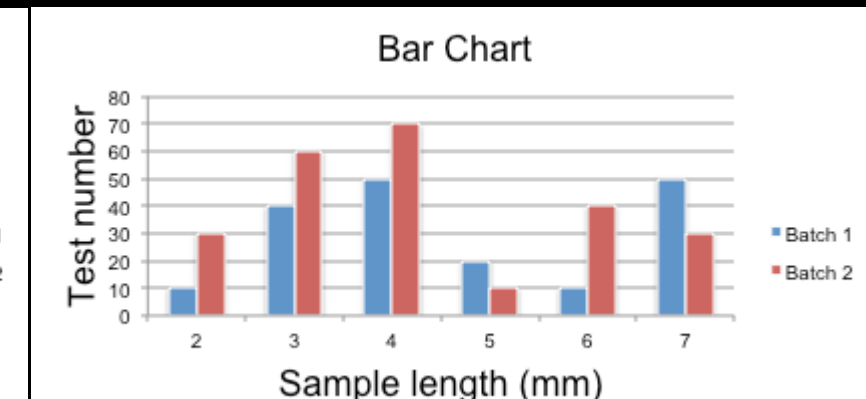
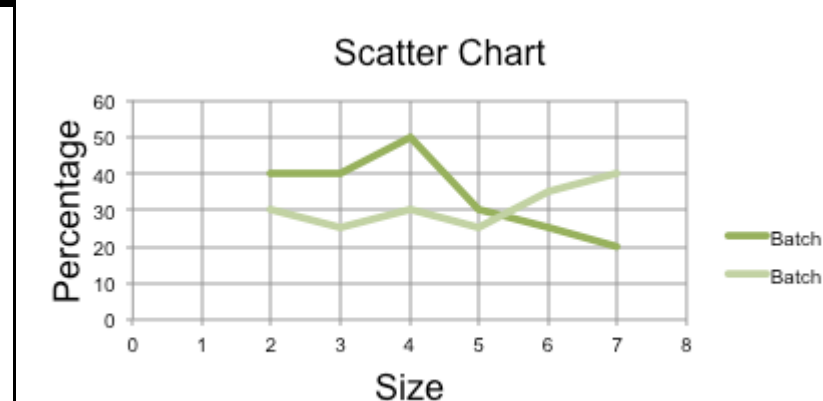
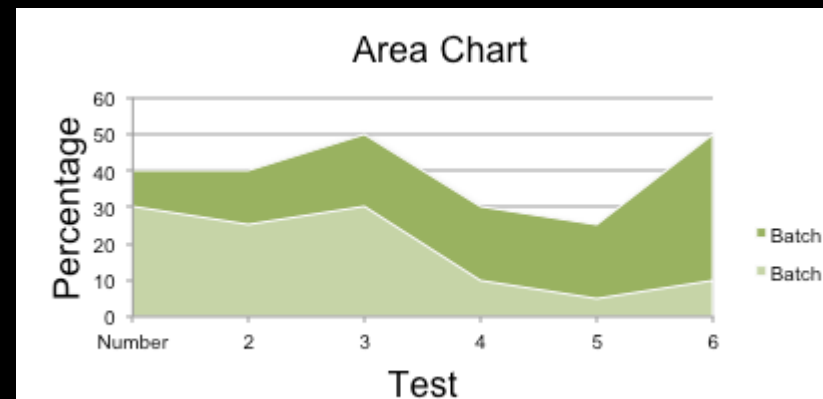
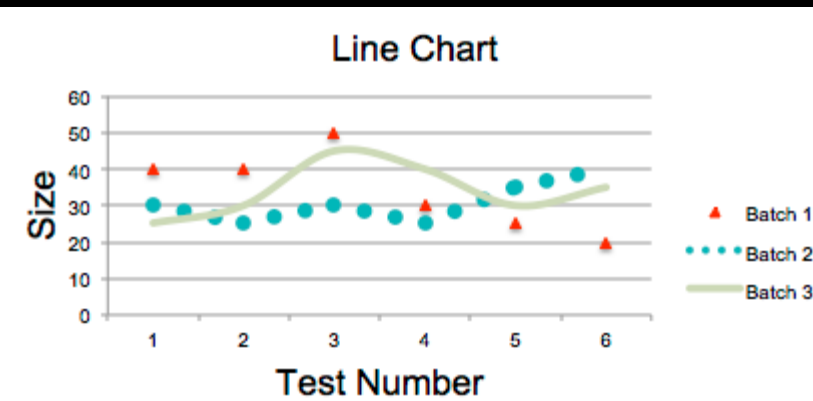
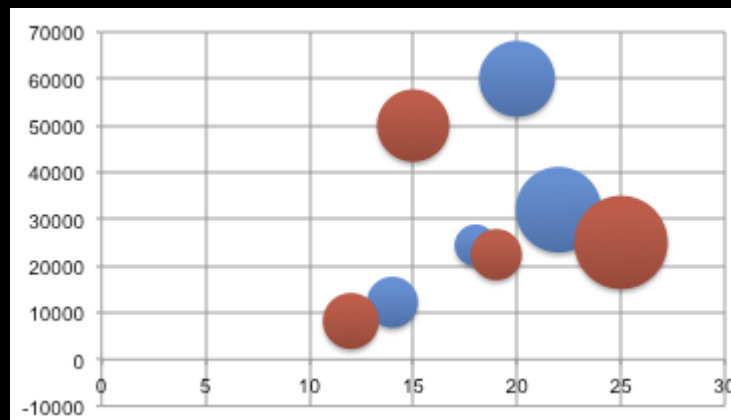
# 메서드를 이용한 순회
for row in ws.iter_rows(min_row=1, max_row=ws.max_row, max_col=3):
    ...
```

- Worksheet의 각 셀 구분자를 이용하여 Cell 객체 선택 가능
 - 객체에 직접 값을 대입하면 해당 객체의 값(**Cell.value**)을 바꿈
- Worksheet 슬라이싱 가능
 - 슬라이스의 마지막을 반드시 정의해야 함
 - 순회시 행, 열 순서로 순회
- 마지막 행렬을 자동으로 찾아 순회하는 경우는 **iter_rows, iter_cols** 메서드 사용
 - **Worksheet.max_row, Worksheet.max_col**로 최대를 명시할 수 있음



- 차트를 만들기 위해서 기본적으로 하나의 차트 객체와 최소 둘 이상의 **Reference** 객체 필요
 - 차트 객체: 차트 구성에 필요
 - **Reference** 객체: 최소 데이터, 범주 두 용도로 필요
- 차트마다 실제 구성 방법은 상이할 수 있음

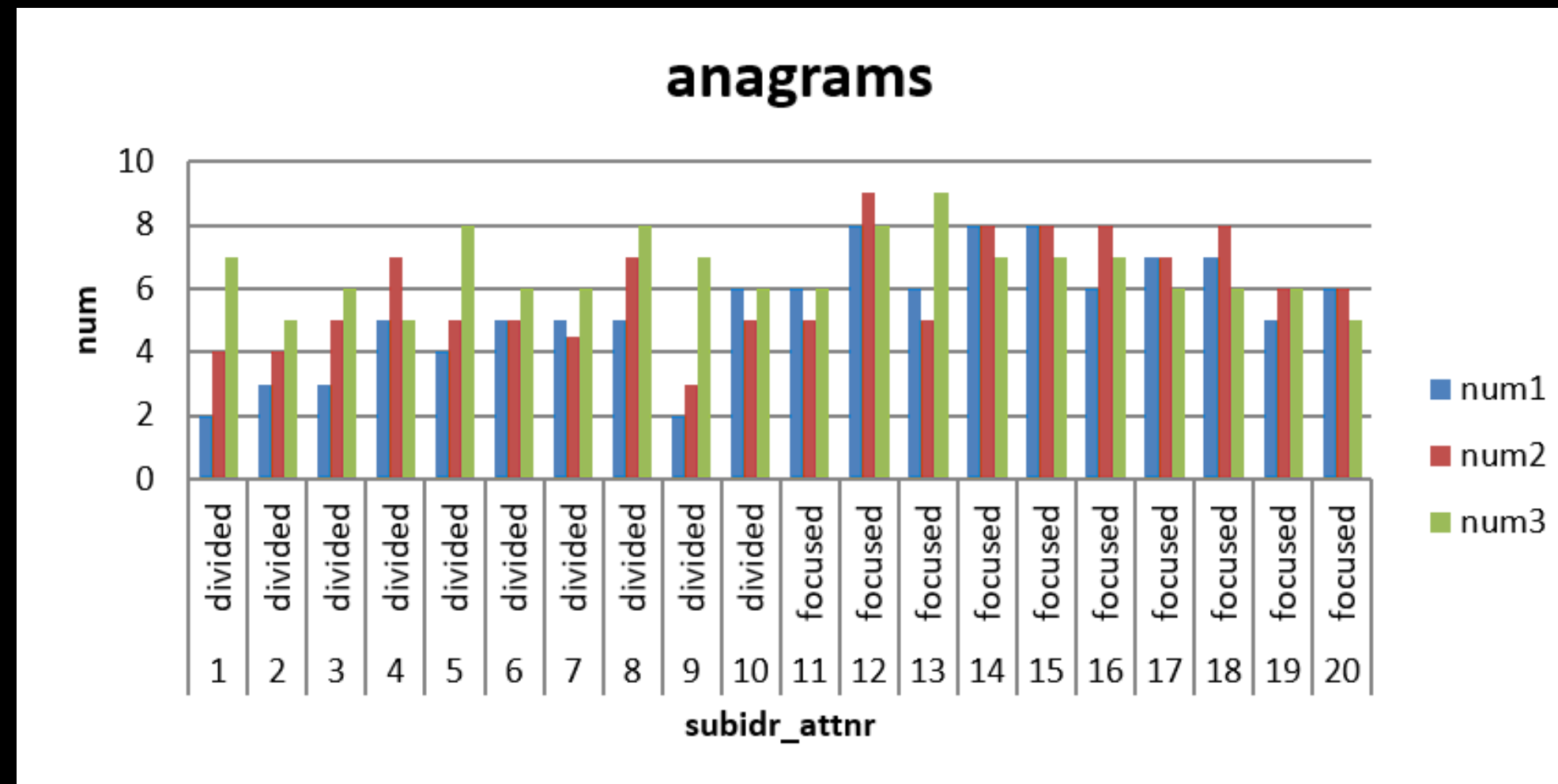
python-pptx로 만들 수 있는 다양한 종류의 차트





	A	B	C	D	E
1	subidr	attnr	num1	num2	num3
2	1	divided	2	4	7
3	2	divided	3	4	5
4	3	divided	3	5	6
5	4	divided	5	7	5
6	5	divided	4	5	8
7	6	divided	5	5	6
8	7	divided	5	4.5	6
9	8	divided	5	7	8
10	9	divided	2	3	7
11	10	divided	6	5	6
12	11	focused	6	5	6
13	12	focused	8	9	8
14	13	focused	6	5	9
15	14	focused	8	8	7
16	15	focused	8	8	7
17	16	focused	6	8	7
18	17	focused	7	7	6
19	18	focused	7	8	6
20	19	focused	5	6	6
21	20	focused	6	6	5

차트를 만들려는 데이터



데이터를 바탕으로 만들어진 차트



```
from openpyxl import load_workbook
from openpyxl.chart import BarChart, Reference
```

```
# 기존 Excel 파일 불러오기
```

```
wb = load_workbook("anagrams.xlsx")
```

```
ws = wb.active
```

```
# 바 형태의 차트 생성
```

```
chart = BarChart()
```

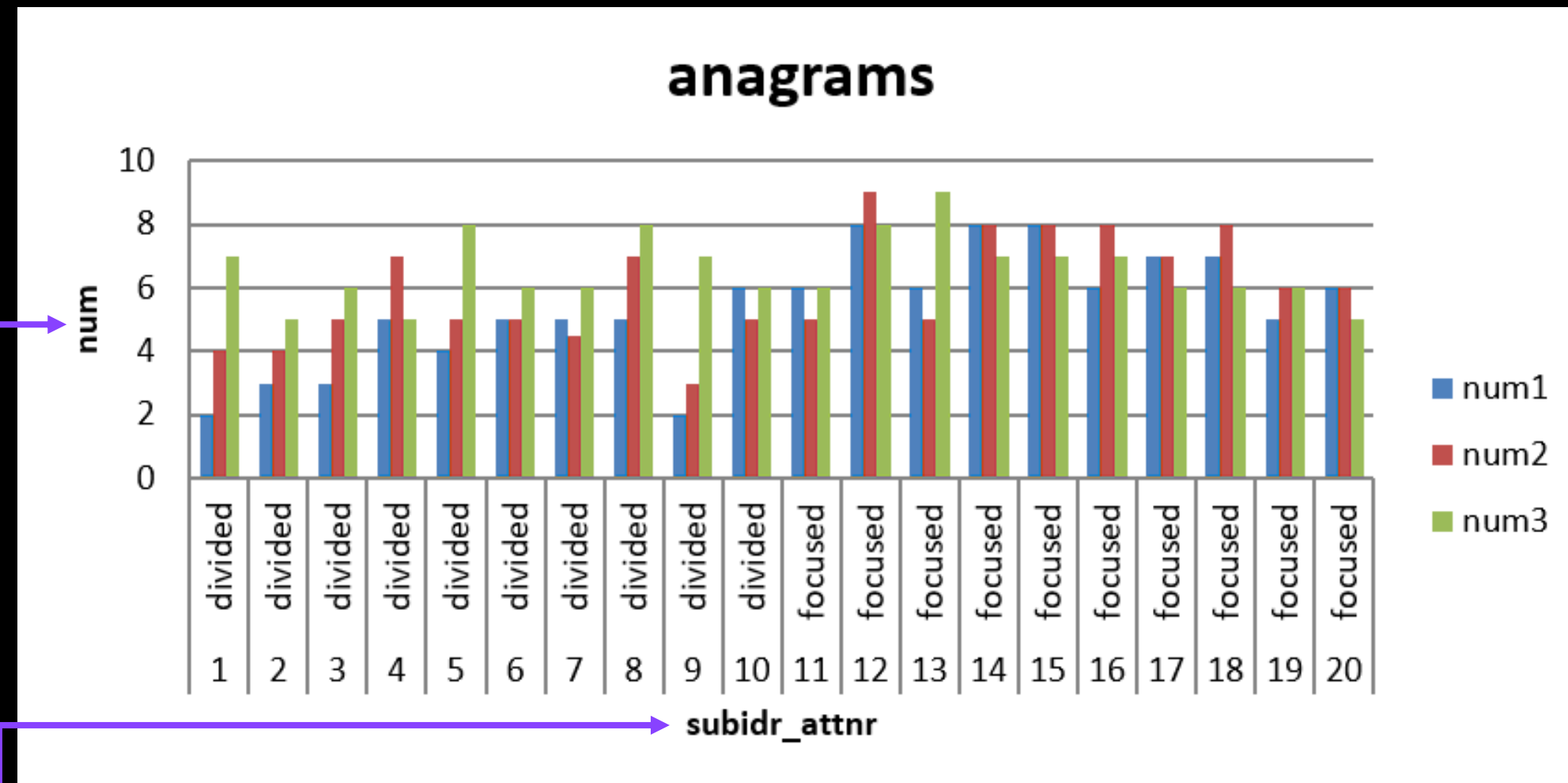
```
# 수직 방향
```

```
chart.type = "col"
```

```
chart.title = "anagrams"
```

```
chart.y_axis.title = "num"
```

```
chart.x_axis.title = "subidr_attnr"
```



- **BarChart** 생성 후 **type**을 "col" (행) 방향으로 설정
- **y** 축과 **x** 축 제목을 표와 반대로 설정
 - 범주가 있는 열이 x 축
 - 데이터가 있는 열이 y 축



```
# 데이터의 위치는 3-5행, 2-21열
# 1열은 계열 이름을 위해 추가
data = Reference(ws, min_col=3, min_row=1, max_col=5, max_row=21)
# 범주의 위치는 1-2행, 2-21열
categories = Reference(ws, min_col=1, min_row=2, max_col=2, max_row=21)

# 차트에 데이터 추가, 1열을 계열 이름으로 사용
chart.add_data(data, titles_from_data=True)
# 차트에 범주 추가
chart.set_categories(categories)

# F1 위치에 차트 배치
ws.add_chart(chart, "F1")

wb.save("document.xlsx")
```

- 표를 바탕으로 데이터 위치 한정
 - 예시의 경우 3행 2열부터 5행 마지막 열까지
 - 첫 번째 열은 계열 이름으로 사용하기 위해 추가
- 표를 바탕으로 범주 위치 한정
 - 예시의 경우 1행 2열부터 2행 마지막 열까지
- **BarChart**의 **add_data** 및 **set_categories** 메서드를 이용하여 데이터와 범주 추가
 - **add_data**에서 **titles_from_data**를 True로 설정하면 첫 번째 열을 범례의 계열 이름으로 사용

4. python-pptx

- Python을 이용한 Microsoft의 PowerPoint 파일 조작 라이브러리
- PowerPoint 파일 제작 및 수정
- PowerPoint 파일의 텍스트 추출



- 설치

- Windows: `py -m pip install python-pptx`
- macOS: `python3 -m pip install python-pptx`

- 임포트

- `import pptx`
- python-pptx가 아님에 유의



간단한 PowerPoint 파일 만들기 예제

Python을 이용한 파일
처리

05 사무용 문서 다루기

4. python-pptx

```
from pptx import Presentation

prs = Presentation()
title_slide_layout = prs.slide_layouts[0]
slide = prs.slides.add_slide(title_slide_layout)
title = slide.shapes.title
subtitle = slide.placeholders[1]

title.text = "Hello, World!"
subtitle.text = "python-pptx was here!"

prs.save("test.pptx")
```

Hello, World!

python-pptx was here!

제목과 부제가 있는 간단한 문서 만들기

결과물



- Presentation
 - 프레젠테이션 전체를 구성하는 단위
- SlideLayout
 - 프레젠테이션에 저장된 슬라이드 레이아웃
- Slide
 - 개별 슬라이드 요소, 슬라이드 레이아웃 정보가 있음
- Shape
 - 슬라이드 안의 개체, 해당 개체가 텍스트를 담고 있는지 등 확인 가능
- Placeholder
 - Shape의 한 종류, 일반적으로 내용을 담고 있는 부분이 해당



```
from pptx import Presentation

prs = Presentation("document.pptx")

for slide in prs.slides:
    for shape in slide.placeholders:
        if not shape.has_text_frame:
            continue
        for paragraph in shape.text_frame.paragraphs:
            for run in paragraph.runs:
                print(run.text)
```

• Text의 계층 구조

- Shape.text_frame
- paragraph
- runs

• 하나의 Shape에서 텍스트를 추출하려면 두 번의 반복문을 거치게 됨

-
- Egg, **bacon**, *sausage and spam*.
 - Spam, bacon, sausage and spam.
 - **Spam**, *egg*, spam, spam, bacon and spam.



```
from pptx import Presentation
from pptx.enum.shapes import PP_PLACEHOLDER

prs = Presentation("example.pptx")

for slide in prs.slides:
    # placeholder 반복
    for shape in slide.placeholders:
        if not shape.placeholder_format:
            continue
        # 각 슬라이드의 제목만 출력하기
        if shape.placeholder_format.type == PP_PLACEHOLDER.TITLE:
            print(f"TITLE: {shape.text}")
```

- python-pptx의 **PP_PLACEHOLDER** 구분을 통해 종류별 텍스트 구분이 가능
- **placeholder_format.type** 과의 비교를 통해 구분
- 프레젠테이션 제목: **CENTER_TITLE**
- 프레젠테이션 부제: **SUBTITLE**
- 제목: **TITLE**
- 한 줄 요약 등: **BODY**
- 본문: **OBJECT**



Placeholder 별로 텍스트 가져오기 예제

Python을 이용한 파일
처리

05 사무용 문서 다루기

4. python-pptx

```
from pptx import Presentation
from pptx.enum.shapes import PP_PLACEHOLDER

prs = Presentation("example.pptx")

for slide in prs.slides:
    print(prs.slides.index(slide))
    for shape in slide.placeholders:
        if shape.placeholder_format.type == PP_PLACEHOLDER.CENTER_TITLE:
            print(f"CENTER_TITLE: {shape.text}")
        if shape.placeholder_format.type == PP_PLACEHOLDER.SUBTITLE:
            print(f"SUBTITLE {shape.text}")
        if shape.placeholder_format.type == PP_PLACEHOLDER.TITLE:
            print(f"TITLE: {shape.text}")
        if shape.placeholder_format.type == PP_PLACEHOLDER.OBJECT:
            if not shape.has_text_frame:
                continue
            for paragraph in shape.text_frame.paragraphs:
                for run in paragraph.runs:
                    print(f"OBJECT: {run.text}")
        if shape.placeholder_format.type == PP_PLACEHOLDER.BODY:
            print(f"BODY: {shape.text}")
```



```
from pptx import Presentation
from pptx.dml.color import RGBColor
from pptx.enum.dml import MSO_THEME_COLOR
from pptx.util import Pt

prs = Presentation("example.pptx")

for slide in prs.slides:
    for shape in slide.placeholders:
        if not shape.has_text_frame:
            continue
        # 모든 Placeholder의 글꼴 변경하기
        for paragraph in shape.text_frame.paragraphs:
            for run in paragraph.runs:
                font = run.font
                font.name = "Calibri"
                font.size = Pt(20)
                font.bold = True
                font.italic = None
                # 색상 테마로 변경
                font.color.theme_color = MSO_THEME_COLOR.LIGHT_1
                # 값으로 변경
                font.color.rgb = RGBColor(0x7F, 0x7F, 0x7F)

prs.save("output.pptx")
```

- 글꼴 이름, 크기, 굵기, 기울임, 색상 등을 일괄적으로 변경 가능
- 글꼴 이름: 영문으로 작성
- 크기: **Pt** 클래스로 크기 전달
- 굵게, 기울임꼴: 설정되어 있을 경우 **True**, 아니면 **None**을 가짐
- 색상: **theme_color** 속성을 이용하여 정해진 테마를 이용하거나 **RGBColor** 클래스로 직접 값 설정 가능
 - **RGBColor**를 사용하는 경우 Red, Green, Blue의 값 조합 이용
 - 0x00-0xFF(hex), 0-255(dec) 사이의 값으로 조정
- 속성을 변수에 대입하여 현재 값을 알아낼 수도 있음



- Egg, bacon, sausage and spam.
- Spam, bacon, sausage and spam.
- Spam, egg, spam, spam, bacon and spam.

코드 실행 전

- Egg, bacon, sausage and spam.
- Spam, bacon, sausage and spam.
- Spam, egg, spam, spam, bacon and spam.

코드 실행 후



필요한 부분만 골라서 글꼴 변경하기

Python을 이용한 파일
처리

05 사무용 문서 다루기

4. python-pptx

```
from pptx import Presentation
from pptx.enum.shapes import PP_PLACEHOLDER
from pptx.util import Pt

prs = Presentation("example.pptx")

for slide in prs.slides:
    for shape in slide.placeholders:
        # 본문만 선택
        if (
            shape.placeholder_format.type != PP_PLACEHOLDER.OBJECT
            or not shape.has_text_frame
        ):
            continue
        for paragraph in shape.text_frame.paragraphs:
            for run in paragraph.runs:
                font = run.font
                font.name = "Calibri"
                font.size = Pt(72)
                font.italic = True

prs.save("output.pptx")
```

- 텍스트 종류별 구분 방법을 사용하여 특정 종류의 글꼴 변경이 가능

TITLE

- OBJECT

변경 전

TITLE

- *OBJECT*

변경 후